Determining the quality and condition of the surface of the collection coins, using artificial intelligence (AI)

- Final Project of Course –

by

Juan Diego Aguirre Gómez

Team: QUASAR

BOOTCAMP TALENTO TECH MINTIC COLOMBIA

UNIVERSIDAD DE ANTIOQUIA

IA-INNOVADOR

MEDELLÍN

2025

ACKNOWLEDGMENTS

I would like to thank the Ministry of Information and Communications (MINTIC), the University of Antioquia, and the Temporary University of Antioquia for organizing the bootcamp and creating these types of spaces where we can grow in our profession, access better opportunities in the tech market, and, in turn, contribute to the development of individuals who can make significant contributions to the country.

INDEX

Cover Page 1

Acknowledgments 2

Index 3

Introduction - Problem Description 5

Challenge in relation to the chosen sector 5

State of the art 6

Justification of Relevance 7

Proposed Solution and Technologies to Use 8

Dataset 8

Creation Dataset 9, 10

Process construction Dataset 11

Software Requeriments 12

Hardware required 12

Filtered Images depurated *elimination bad images) 13

General Description of the Solution 13

User gives image – PREPROCESS 13

FILTER make an image filtered and renamed 14

Training ENTRENAMIENTO 14, 15, 16, 17, 18

Deploy the application to the user APP 19

Summary of technologies used 19

AI Techniques Used 19

Preliminary System Architecture 19

Libraries Used 19

Languages and Frameworks 20

Data Sources and Preprocessing Process 21

Step 1: Data Input 21
Step 2: Image Processing 21
Step 3: Model Training 23
Step 4: Model Prediction 24
Step 5: Output 24
AI Module: Algorithms and AI Services Implemented 24
User Interface: Prototype in Streamlit 24
Integrations and Information Flows 24
Functional Prototype at Streamlit 25
Objective of the Prototype 25
Code Structure 25
Examples of Implemented Functionalities 25
Future improvements for this work 28
Innovations in this project 29
Monetization possibilities 29
Hardware used - Device Characteristics 29, 30
Performance of the used machine 31
Filter 31
Training 32
Vocabulary 33
Software, document and video REPOSITORIES 34

Determining the quality and condition of the surface of the collection coins, using artificial intelligence (AI)

Introduction

Problem Description

Coin collectors' exchange processes face challenges in agreeing on the condition of the coins at the time of a trade or sale, making transactions difficult and making it less enjoyable to enjoy the hobby.

This is a project of technological innovation.

Challenge in relation to the chosen sector.

Current methods for evaluating coin condition often rely on subjective opinions of each person within the interchange, leading to disputes and mistrust between collectors. There is a need for a more accurate, standardized, and automated approach to assess coin quality and account for the condition of wear.

In the coin collecting sector, their value is heavily influenced by their condition. The inability to reach a consensus on the state of a coin can hinder transactions and create friction among collectors. This AI solution aims to facilitate arbitration between parties by providing an accurate and unbiased assessment of coin condition, thereby enhancing trust and efficiency among the participants of this hobby.

List of qualities used in numismatic to indicate the wear on coins. Are eight (8):

Letras	English	Español	Valor Aprox
PROOF	**Proof** Blank	Prueba en Blanco	160%
UNC	**U**ncirculate	Sin Circular	120%
XF	**E**xtremely **F**ine	Extremadamente Bien	100% vr comercio
VF	**V**ery **F**ine	Muy Bien	85%
F	**F**ine	Bien	70%
VG	**V**ery **G**ood	Muy Buena	55%
G	**G**ood	Buena	45%
PR	**P**oor	Pobre	20%

Valor Aprox in the last image: depending of the general value (commercial value, not the denomination in the coin) on a market of a specific coin with quality: **XF**, the value for the collectors will be 160% if is proof, 120%, and so on.

Examples of the same coin with different steps of wear.

State of the art

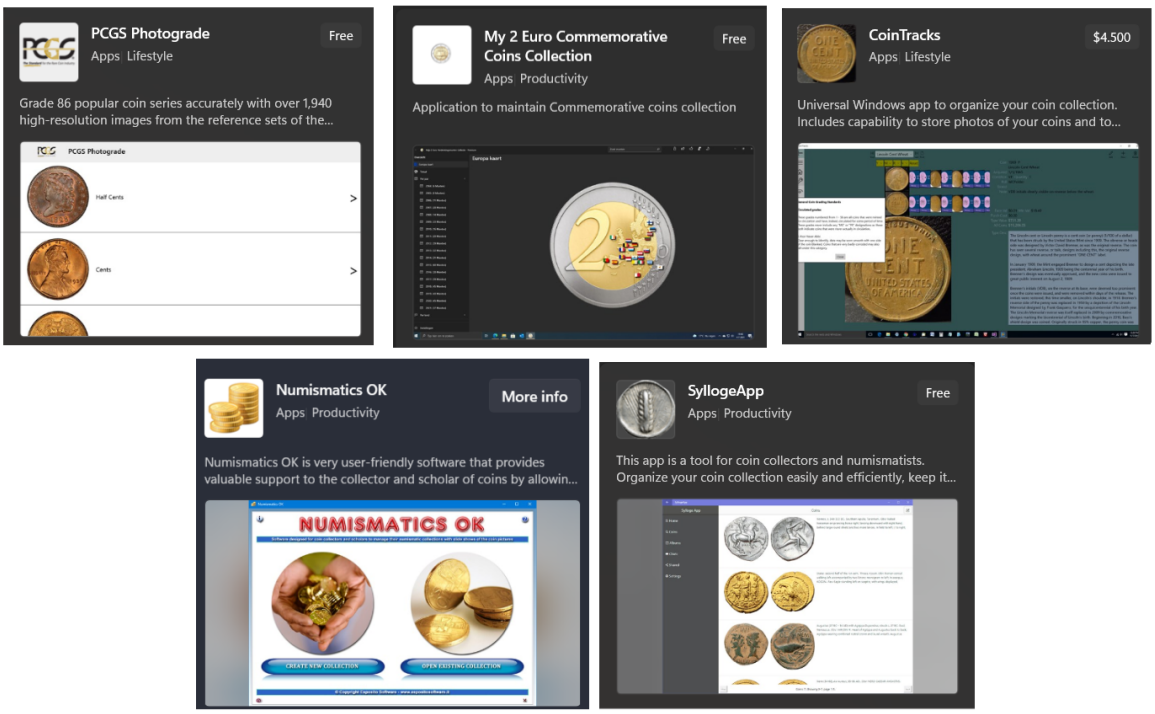
Currently, there are many free and paid mobile apps, available in all languages, to identify and organize a coin collection. They are very accurate and fast, and provide information about the country, date, denomination, and market value of the coin. However, there are no apps that determine the wear status of a coin, a factor that directly affects its market or exchange value.

Some current applications:

CoinSnap

Heritcoin

Coinscope



There are many cell phone apps to identify coins and organize collections, but not to indicate the state of deterioration.

Justification of Relevance

In the context of coin collecting, where the condition of coins significantly influences their value, implementing solutions that rely on precise and automated data can revolutionize the way collectors assess and trade coins. By ensuring that the professional concept used for evaluating coin conditions is consistent, we can enhance the reliability of assessments, thereby fostering trust among collectors and facilitating smoother transactions.

Moreover, the potential impact of this innovation extends beyond the realm of coin collecting. As industries increasingly embrace digital transformation, the methodologies developed for assessing the surface of metallic condition can be adapted to other sectors, as in astronomy with image analysis and in microbiology with the segmentation and identification of elements in the samples others areas: Numismatic, optometry, electronics, Production Industrial in line, botany, Video games, etc.

Collectors will spend less time negotiating and more time engaging in meaningful transactions, ultimately driving growth in the market in terms of culture, asset conservation, and historical information. Additionally, the adoption of standardized assessments can lead to a digital transformation in the coin collecting community, encouraging more collectors to participate in the hobby and fostering a culture of transparency and trust.

Proposed Solution and Technologies to Use

General Description of the Solution:

The proposed solution involves the implementation of a computer vision service designed to accurately assess the condition of coins during exchanges among collectors. This service will utilize advanced image processing techniques to analyze coin images, providing objective evaluations of their quality based on established grading standards. By automating the assessment process, we aim to eliminate discrepancies and foster trust between collectors, ultimately streamlining the trading process.

Involved Technologies:

Computer Vision - 2D

Convolucional Network

Clasificación

Regression model (numerical answer from 0 to 1)

DATASET

After look for datasets in this alternatives:

1. **UCI Machine Learning Repository**

   A comprehensive collection of datasets for machine learning research.

2. **Registry of Open Data on AWS**

   A platform hosting various datasets from organizations like NASA and NIH.

3. **Data.gov**

   A catalog of U.S. government datasets across multiple domains.

4. **Awesome Public Datasets**

   A curated list of high-quality public datasets across various topics.

5. **List of Datasets for Machine Learning Research**

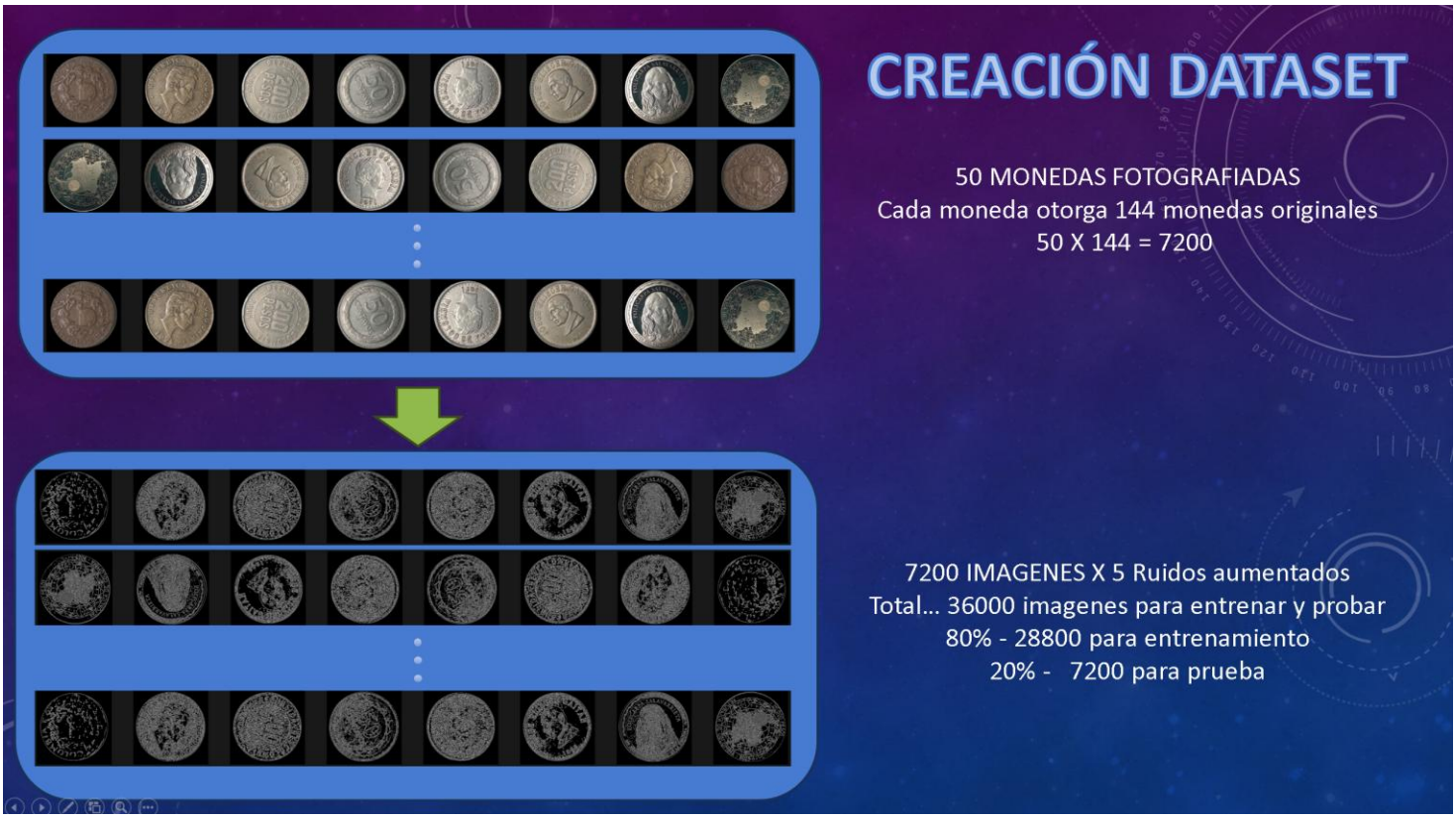
   A Wikipedia page compiling various datasets for machine learning tasks.

6. **Kaggle**

7. **Google Dataset Search**

The answer is clear… There is not available database of images of coins to use at internet, paying or not, to use the deterioration of coins. Then… let's make it.

We need to make a new dataset



The labels will put for the professional numismatic collectors of Medellín, Colombia.

Now, to make all the process will work three (3) steps:

1) Filter: to reduce the image size, found the circle in the image, crop and remove the background.
2) Training: The images obtained from the filter process, pass to the train the model.
3) Display: The deploy to the user the app for give the image, and obtained a predict.

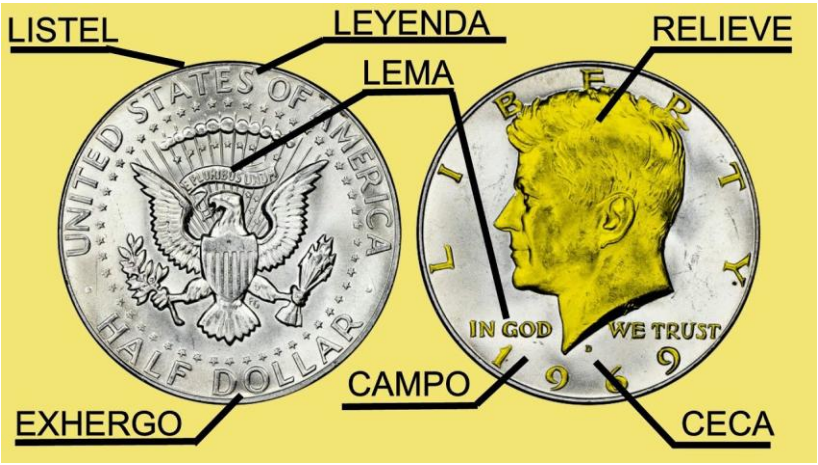
   Note that in this process, we are not making the three processes at time, and the user in this occasion will give to the training model a filter image just processed. To the future this will be a improve for this application.

Then, there are only three Python files to make all the steps in the process (Are three (3)):

1) The **filtroHSV.py** file primarily utilizes the OpenCV library (cv2) for image processing tasks, specifically for color space transformation, image resizing, and circle detection. The techniques used in this file include:

- **Color Space Transformation**: Converting images to the HSV color space.
- **Image Processing Techniques**: Resizing, blurring, and detecting circles in images.
- **Masking and Thresholding**: Creating masks for specific regions and applying thresholding to highlight features.

Images with false color. Note how the field of the coin is green.



Campo = Field

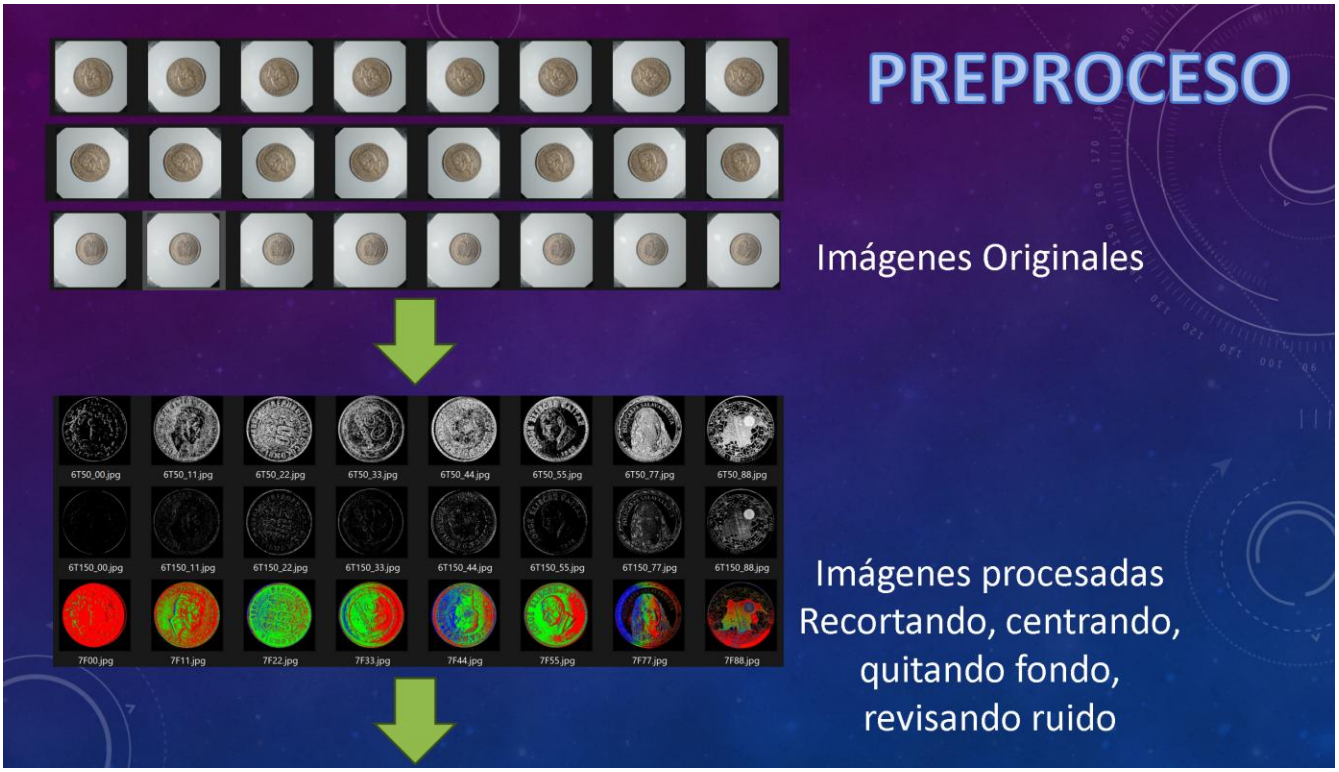
Input image preparation process

Original image

gray image and resize

image
without background

With these steps, we reduce the quantity of pixels to make easier the process reducing the quantity of pixels to work.



follow the training

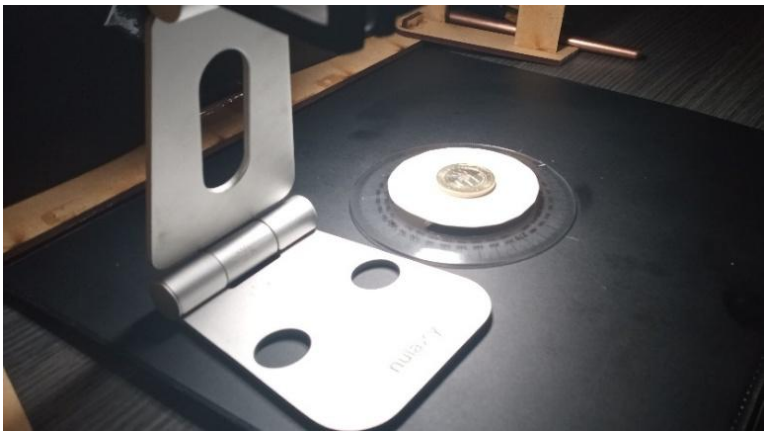
SOFTWARE REQUIREMENTS: Python 3.11.6 and their library versions used in the environment (.venv):

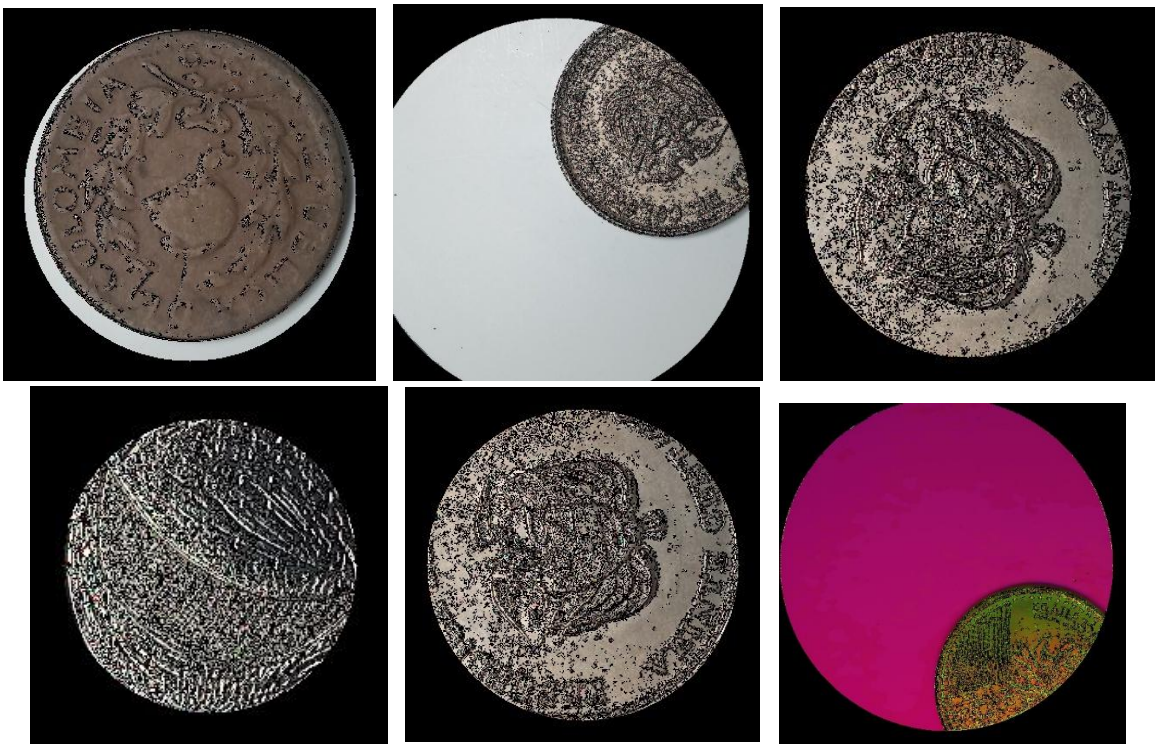
1. numpy==1.24.3
2. opencv-python==4.8.1.78
3. tensorflow==2.15.0
4. keras==2.15.0
5. pillow==10.2.0
6. streamlit==1.43.2
7. matplotlib==3.10.1
8. seaborn==0.12.2
9. scikit-learn==1.6.1
10. ipykernel==6.21.2

This is the list on ‘requirements.txt’ for the three Python files used.

Was necessarily make a turntable or little machine with light and mechanism to spin the coin to take the photography with a cellphone (very good machine to take photos, additionally the users will use this device to take the images:

After this process were required to remove some incorrect filters masked like this to avoid troubles at the training moment:





These coins don't have these aviaries on the surface, this is the result of the digital treatment.

General Description of the Solution

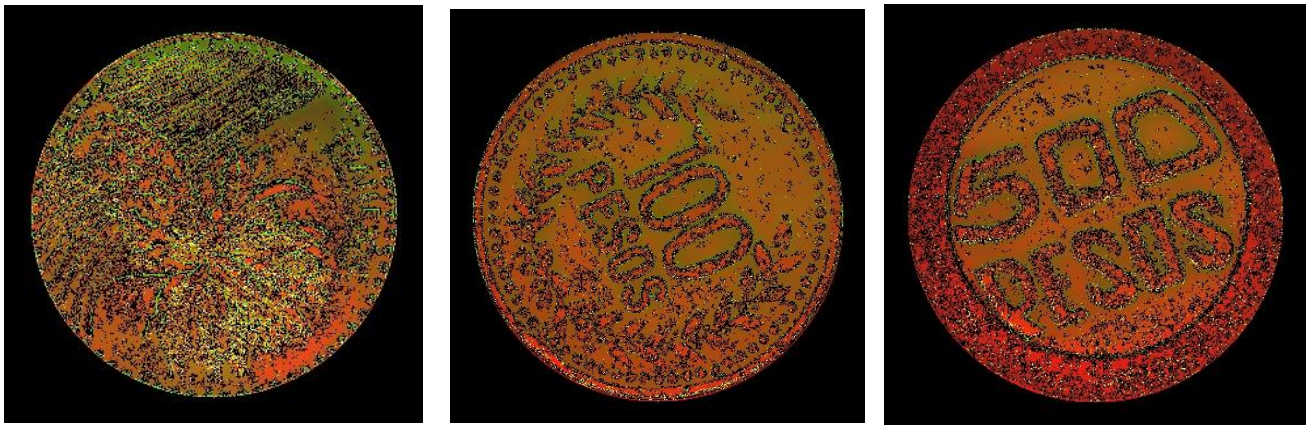
User gives image – PREPROCESS OF THE RECEIVED IMAGES

The user will give an image like this:

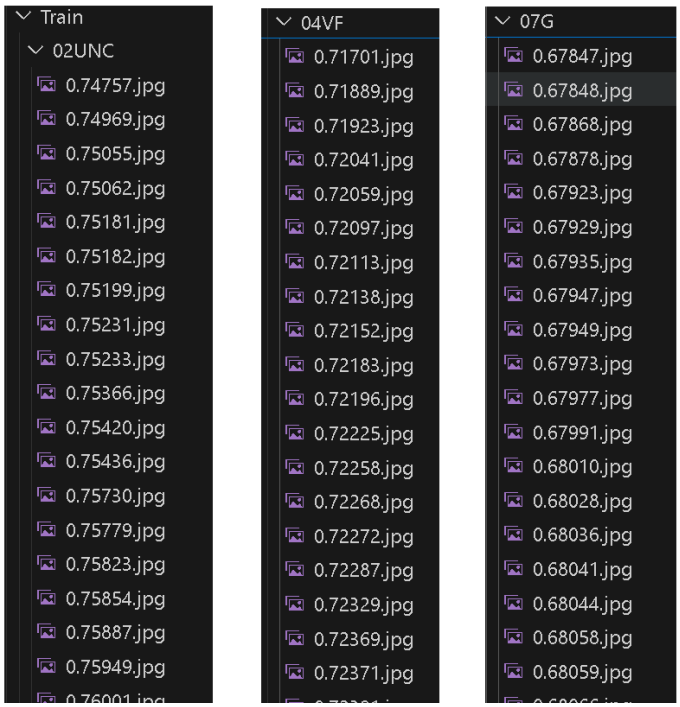
With margins, with background and in a huge size (3120 x 3120 pixels).
Normally these images coming from cellphones of the users.
After the filter process, the result is an image like this with filters, without background (around the coin), and resized (320 x 320 pixels in this case):

FILTER Laplacian with threshold 50 and 150 and HSV

Now the images noise are visible for the software.



While the process is made, the code is asking for the proportion of RED pixels inside the circle, respect the total pixels of the image, and this data is used to give the name to the image file. This, maybe is a wonderful idea or an innovation to give more data to train the model, because we are given information outside of the image and taking advance using the file name to give information.



2) The **entrenamiento.py** file utilizes several libraries and techniques related to machine learning and deep learning, specifically:

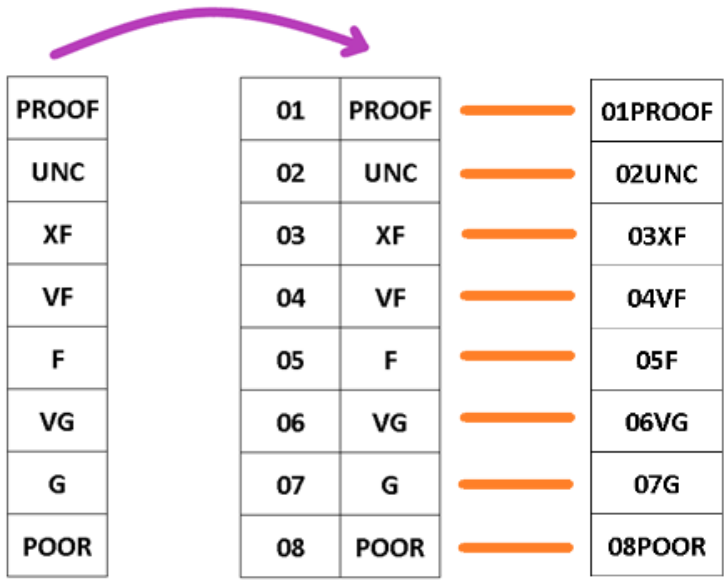
- **Libraries Used**:

- **TensorFlow**: For building and training the deep learning model.
- **OpenCV**: For image processing tasks.
- **NumPy**: For numerical operations.
- **Matplotlib and Seaborn**: For data visualization.
- **Scikit-learn**: For metrics and model evaluation.

- **AI Techniques**:
  - **Convolutional Neural Networks (CNNs)**: The model is built using a sequential CNN architecture for classifying coin quality.
  - **Data Augmentation**: Using ImageDataGenerator to preprocess images and create training and validation datasets.
  - **Model Training and Evaluation**: The script includes training the model, evaluating its performance, and visualizing results such as confusion matrices and training history.

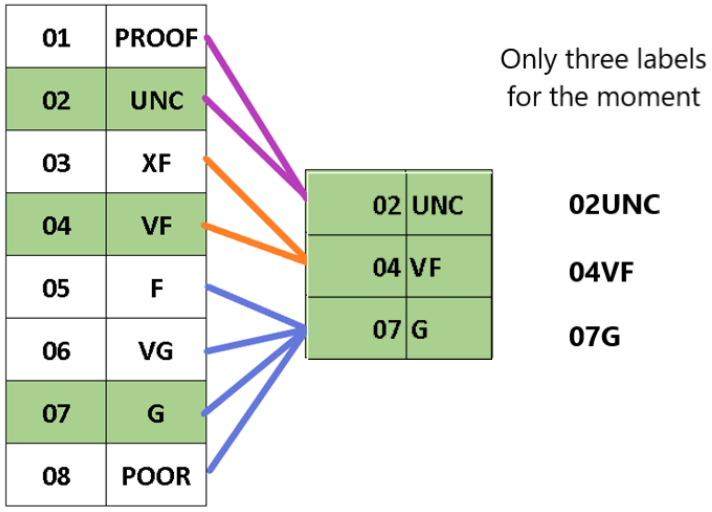
CHARACTERISTICS AND LABELS:

We enumerated the order of the qualities in numismatic, to avoid the system confuse with the alphabetical order:



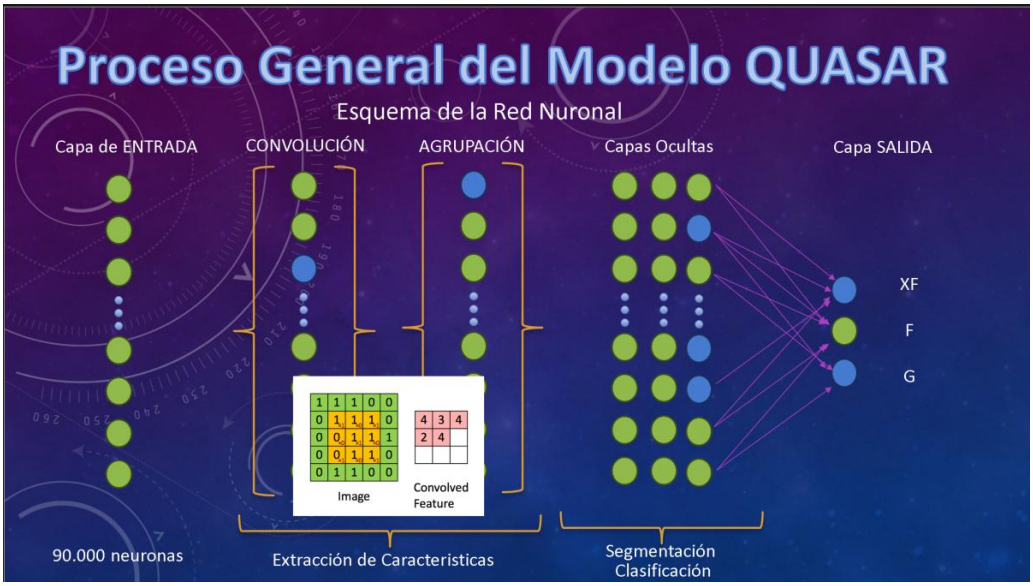
See tha page five (5) in this document to remember the numismatic classification of wears.

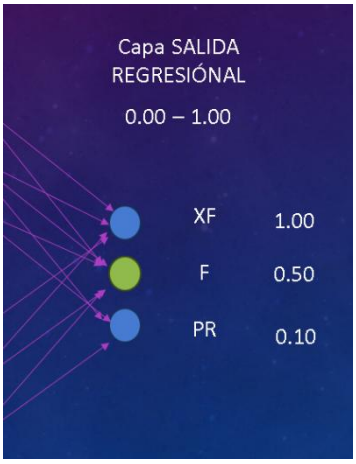
This training will use for the moment only three (3) characteristics or label of the eight (8) possibles, for avoid efforts and waste of energy while the project grown, it is a way to be clear how the structure will work and do not spend time and energy with many details of fine characteristics:



This project only will work for the moment with three characteristics, the improves for future will be to detect all the eight labels. This is important to deliver this work at time, because we only have four (4) weeks to make all the process and write this document in this course. The other hand, we will save many time to prove the structure and functioning of this software is able.

CNN - DENSE NEURAL NETWORK will be use in this work, convolutional type.





90,000 input neurons for 300 x 300 pixels images.

Capture of the training moment. - 1

```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE   COMMENTS

11/11 [==============================] - ETA: 0s - loss: 1.7584 - accuracy: 0.3621D:\TalentoTech2\Quasar\Python\CrearDataSet\Entrenam
e-packages\keras\src\engine\training.py:3103: UserWarning: You are saving your model as an HDF5 file via `model.save()`. This file fo
mend using instead the native Keras format, e.g. `model.save('my_model.keras')`.
  saving_api.save_model(
11/11 [==============================] - 17s 1s/step - loss: 1.7584 - accuracy: 0.3621 - val_loss: 1.0860 - val_accuracy: 0.3333
Epoch 2/30
11/11 [==============================] - 16s 1s/step - loss: 1.0151 - accuracy: 0.5115 - val_loss: 0.7735 - val_accuracy: 0.7262
Epoch 3/30
11/11 [==============================] - 16s 1s/step - loss: 0.4622 - accuracy: 0.8218 - val_loss: 0.0587 - val_accuracy: 1.0000
Epoch 4/30
11/11 [==============================] - 16s 1s/step - loss: 0.1415 - accuracy: 0.9454 - val_loss: 0.2805 - val_accuracy: 0.8452
Epoch 5/30
11/11 [==============================] - 15s 1s/step - loss: 0.0993 - accuracy: 0.9684 - val_loss: 0.0597 - val_accuracy: 0.9762
Epoch 6/30
11/11 [==============================] - 16s 1s/step - loss: 0.0687 - accuracy: 0.9684 - val_loss: 0.1896 - val_accuracy: 0.8929
Epoch 7/30
11/11 [==============================] - 15s 1s/step - loss: 0.0393 - accuracy: 0.9914 - val_loss: 0.0807 - val_accuracy: 0.9643
Epoch 8/30
11/11 [==============================] - 15s 1s/step - loss: 0.0069 - accuracy: 1.0000 - val_loss: 0.0048 - val_accuracy: 1.0000
Epoch 9/30
11/11 [==============================] - 16s 1s/step - loss: 0.0134 - accuracy: 0.9971 - val_loss: 0.0097 - val_accuracy: 1.0000
Epoch 10/30
11/11 [==============================] - 16s 1s/step - loss: 0.0017 - accuracy: 1.0000 - val_loss: 0.0248 - val_accuracy: 0.9881
Epoch 11/30
11/11 [==============================] - 16s 1s/step - loss: 0.0034 - accuracy: 1.0000 - val_loss: 0.0023 - val_accuracy: 1.0000
Epoch 12/30
11/11 [==============================] - 16s 1s/step - loss: 4.6350e-04 - accuracy: 1.0000 - val_loss: 0.0293 - val_accuracy: 0.9881
```

Capture of the training moment. - 2

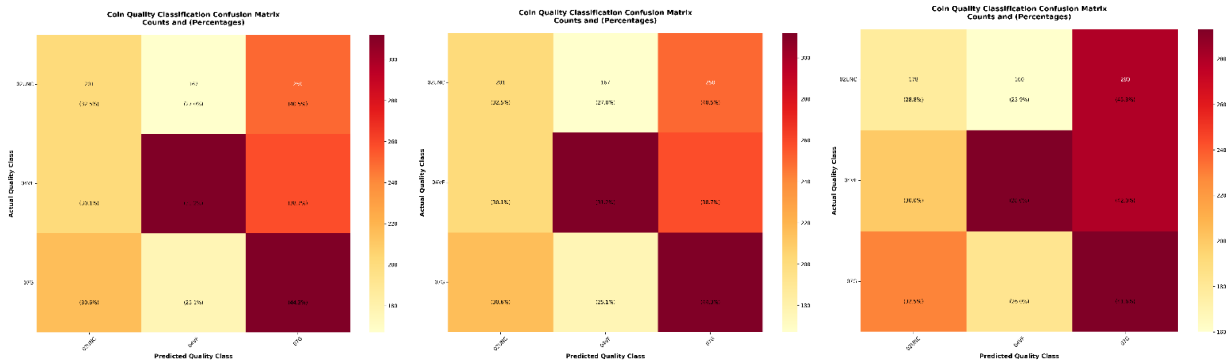
```
PROBLEMS 4   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE   COMMENTS

11/11 [==============================] - 14s 1s/step - loss: 0.2472 - accuracy: 0.8966 - val_loss: 0.4114 - val_accuracy: 0.8095
Epoch 17/30
11/11 [==============================] - 13s 1s/step - loss: 0.2448 - accuracy: 0.8966 - val_loss: 0.3281 - val_accuracy: 0.8929
Epoch 18/30
11/11 [==============================] - 13s 1s/step - loss: 0.1815 - accuracy: 0.9282 - val_loss: 0.2881 - val_accuracy: 0.9048
Epoch 19/30
11/11 [==============================] - 13s 1s/step - loss: 0.1915 - accuracy: 0.9253 - val_loss: 0.2541 - val_accuracy: 0.8929
Epoch 20/30
11/11 [==============================] - 14s 1s/step - loss: 0.1324 - accuracy: 0.9483 - val_loss: 0.1475 - val_accuracy: 0.9286
Epoch 21/30
11/11 [==============================] - 13s 1s/step - loss: 0.1363 - accuracy: 0.9454 - val_loss: 0.1701 - val_accuracy: 0.9167
Epoch 22/30
11/11 [==============================] - 13s 1s/step - loss: 0.1643 - accuracy: 0.9339 - val_loss: 0.1642 - val_accuracy: 0.9286
Epoch 23/30
11/11 [==============================] - 13s 1s/step - loss: 0.1297 - accuracy: 0.9368 - val_loss: 0.2576 - val_accuracy: 0.8690
Epoch 24/30
11/11 [==============================] - 14s 1s/step - loss: 0.1140 - accuracy: 0.9626 - val_loss: 0.1527 - val_accuracy: 0.9643
Epoch 25/30
11/11 [==============================] - 13s 1s/step - loss: 0.0978 - accuracy: 0.9598 - val_loss: 0.3844 - val_accuracy: 0.8690
Epoch 26/30
11/11 [==============================] - 13s 1s/step - loss: 0.1340 - accuracy: 0.9540 - val_loss: 0.1245 - val_accuracy: 0.9524
Epoch 27/30
11/11 [==============================] - 13s 1s/step - loss: 0.1443 - accuracy: 0.9483 - val_loss: 0.2999 - val_accuracy: 0.8810
Epoch 28/30
11/11 [==============================] - 13s 1s/step - loss: 0.1214 - accuracy: 0.9483 - val_loss: 0.2111 - val_accuracy: 0.9405
Epoch 29/30
11/11 [==============================] - 13s 1s/step - loss: 0.0983 - accuracy: 0.9598 - val_loss: 0.2636 - val_accuracy: 0.8929
Epoch 30/30
 8/11 [====================>.........] - ETA: 3s - loss: 0.0654 - accuracy: 0.9722
```

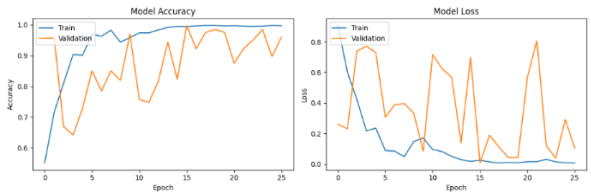
Capture of the training moment. - 3

```
41/41 [==============================] - 58s 1s/step - loss: 1.2031 - accuracy: 0.5089 - val_loss: 0.3311 - val_accuracy: 0.8474
Epoch 2/40
41/41 [==============================] - 61s 1s/step - loss: 0.4707 - accuracy: 0.8218 - val_loss: 0.3630 - val_accuracy: 0.7009
Epoch 3/40
41/41 [==============================] - 58s 1s/step - loss: 0.3683 - accuracy: 0.8745 - val_loss: 0.2077 - val_accuracy: 0.9252
Epoch 4/40
41/41 [==============================] - 57s 1s/step - loss: 0.4024 - accuracy: 0.8629 - val_loss: 0.2470 - val_accuracy: 0.8536
Epoch 5/40
41/41 [==============================] - 60s 1s/step - loss: 0.2405 - accuracy: 0.9311 - val_loss: 0.3402 - val_accuracy: 0.8411
Epoch 6/40
41/41 [==============================] - 59s 1s/step - loss: 0.1103 - accuracy: 0.9675 - val_loss: 0.3422 - val_accuracy: 0.8193
Epoch 7/40
41/41 [==============================] - 58s 1s/step - loss: 0.0754 - accuracy: 0.9814 - val_loss: 0.2009 - val_accuracy: 0.8972
Epoch 8/40
41/41 [==============================] - 56s 1s/step - loss: 0.0521 - accuracy: 0.9853 - val_loss: 0.4342 - val_accuracy: 0.8411
Epoch 9/40
41/41 [==============================] - 56s 1s/step - loss: 0.0615 - accuracy: 0.9706 - val_loss: 0.0398 - val_accuracy: 0.9938
Epoch 10/40
41/41 [==============================] - 55s 1s/step - loss: 0.0395 - accuracy: 0.9853 - val_loss: 0.3883 - val_accuracy: 0.8629
Epoch 11/40
41/41 [==============================] - 55s 1s/step - loss: 0.0302 - accuracy: 0.9907 - val_loss: 0.1211 - val_accuracy: 0.9408
Epoch 12/40
```

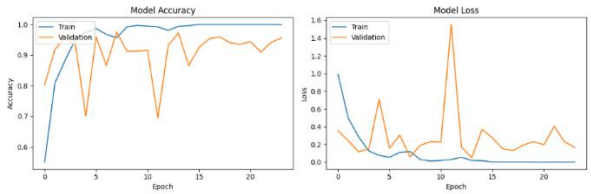
Were used only 40 Epochs, but the system only uses 25 or 30



Three different trainings gave similar confusion matrixes. Maybe the problem is at the noise filter used.



Training the validation are not stable.



This training was better. When we looked for the estrange top of the validation at 11 epoch, we found a bad filter image made.

3) The **app.py** file is responsible for creating a web interface using Streamlit to display the quality of coins based on the trained model. Here are the key points regarding the libraries, languages, and frameworks used:

- **Libraries Used**:
  - **TensorFlow/Keras**: For loading the trained model and making predictions.
  - **NumPy**: For numerical operations and image array manipulations.
  - **PIL (Pillow)**: For image handling and preprocessing.
  - **Streamlit**: For creating the web application interface.
- **AI Techniques**:
  - **Model Loading and Prediction**: The application loads a pre-trained model and uses it to classify uploaded images of coins.
  - **Image Preprocessing**: The uploaded images are resized and normalized to match the input shape of the model before making predictions.
  -

Summary of technologies used

AI Techniques Used:

- **Image Processing**: Techniques such as color space transformation, resizing, and circle detection are employed in the filtroHSV.py file.
- **Convolutional Neural Networks (CNNs)**: The entrenamiento.py file implements a CNN model for classifying coin quality.
- **Data Augmentation**: Utilized in the training process to enhance the dataset.
- **Model Evaluation**: Includes metrics like accuracy and confusion matrices to assess model performance.

Libraries Used:

- **OpenCV**: For image processing tasks (e.g., cv2).
- **TensorFlow/Keras**: For building, training, and loading the deep learning model.
- **NumPy**: For numerical operations and handling image data.
- **Matplotlib and Seaborn**: For visualizing results and training history.

- **Scikit-learn**: For model evaluation metrics.

Languages and Frameworks:

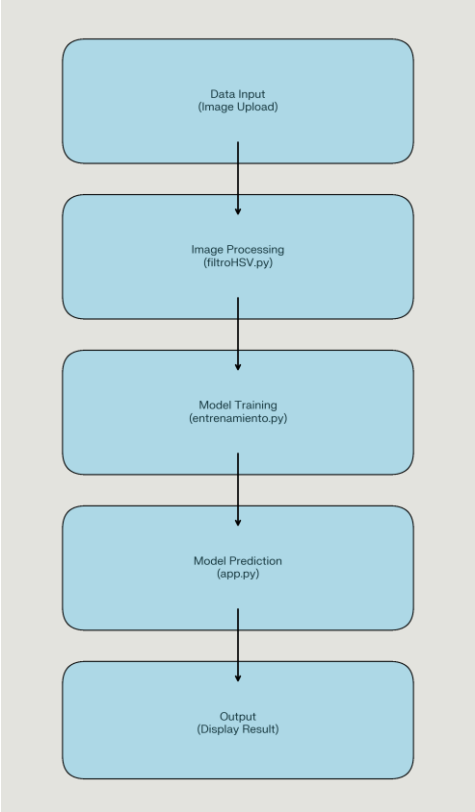
- **Python**: The primary programming language used for the project.
- **Streamlit**: A framework for creating the web application interface to display predictions.

By combining these technologies, we aim to create a not the best robust and not the most efficient solution that improves the accuracy of coin condition assessments but it is the initial born of the project.

This is a baby project, that it will improve the characteristics and the technics, day by day. In other words this is the version QUASAR 0.01 (Quasar is the name of this team).

Description of the main components:

- Conceptual diagram of the solution:



The process described in the diagram can be summarized as follows:

Data Sources and Preprocessing Process

Step 1: Data Input

The workflow begins with the acquisition of data, specifically the upload of images. This step serves as the foundation for subsequent analysis and involves collecting raw visual data from users or systems. These images are prepared for processing to ensure compatibility with the pipeline.

Step 2: Image Processing

Once the images are uploaded, they undergo preprocessing using a script named filtroHSV.py. This phase focuses on refining the raw data by applying techniques such as color space transformation, resizing, and circle detection. The goal is to extract meaningful features and prepare the images for model training.

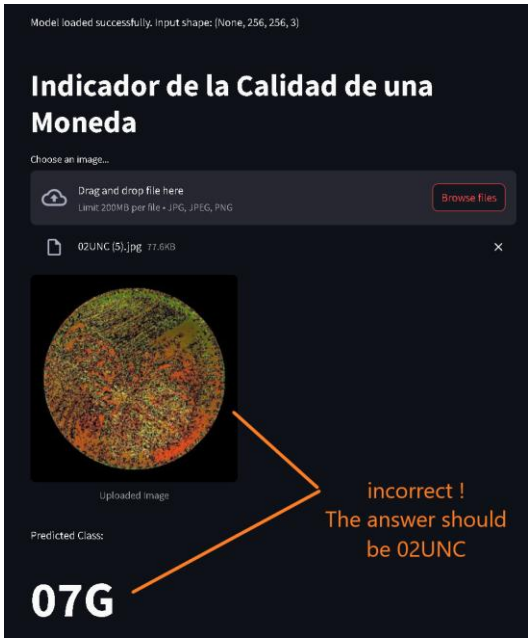
Note: The image given to the system is a preprocessed image, we must remember that the train is made with this kind of image with filters, without margins and without background.

Remember: the user gives an image like this:

With margins, with background and in a huge size. So, the image given for the user is treatment before go to the AI model. This is a PREPROCESS of the imagen.

Others examples of the streamlit process displaying at the web:

Not all is excellent:



The 83% of the processed images that were not used at the training, are correct. This is a very good work for the moment. The troubles maybe are in many thinks, just now we are looking for the structure of work of this system.

Step 3: Model Training

The processed images are then used to train a machine learning model via entrenamiento.py. During this stage, the model learns to identify patterns and relationships within the data using convolutional neural networks (CNNs). Training involves iterative optimization techniques like backpropagation to minimize errors and improve prediction accuracy. Hyperparameters such as learning rate and epochs are adjusted to achieve optimal performance, and data augmentation is utilized to enhance the dataset.

Step 4: Model Prediction

After training, the model is deployed for inference using app.py. In this step, the trained model analyzes new, unseen images and generates predictions based on its learned patterns. This phase represents the practical application of the model's capabilities.

Step 5: Output

Finally, the predictions are displayed to users in a user-friendly format. This output provides actionable insights or results derived from the processed data, completing the pipeline's functionality.

This structured workflow integrates data acquisition, preprocessing, model training, inference, and visualization into a cohesive system for AI-driven image analysis.

- AI Module: Algorithms and AI services implemented

The project implements various algorithms and AI services, including image processing techniques, convolutional neural networks (CNNs) for classification, and data augmentation methods to enhance model performance. These components work together to provide a robust solution for AI-driven image analysis.

- User Interface: Prototype in Streamlit, interactive visualization and analysis.

The project features a prototype built using Streamlit, providing an interactive platform for visualization and analysis of coin quality predictions. Users can upload images, view results, and gain insights through a user-friendly interface.

- Integrations and Information Flows: Communication between modules and tools.

The project facilitates seamless communication between various modules and tools, ensuring efficient data transfer and processing. This includes the flow of images from the user interface to the image

processing module, the interaction between the processing and training modules, and the output of predictions back to the user interface for visualization.

Functional Prototype in Streamlit

- Objective of the Prototype: Demonstrate at least one key feature of the solution (for example, interactive visualization of results or automated data preprocessing).

The prototype aims to demonstrate key features of the solution, such as interactive visualization of coin quality predictions and automated data preprocessing. Users can upload images, view real-time predictions, and gain insights into the model's performance through an intuitive interface.

- Code Structure:

The project is organized into distinct modules, each serving a specific purpose. The main components include:

1. **Image Processing Module (filtroHSV.py): Responsible for preprocessing images, including color space transformation and feature extraction.

2. **Model Training Module (entrenamiento.py):Contains the implementation of the convolutional neural network (CNN) for classifying coin quality, along with data augmentation techniques.

3. **User Interface Module (app.py):Built using Streamlit, this module provides an interactive platform for users to upload images and view predictions.

4. **Integration and Communication**: The modules communicate seamlessly, allowing for efficient data flow from image upload to processing, training, and output visualization.

- Examples of implemented functionalities.

1. **Image Upload**: Users can upload images of coins through the Streamlit interface, allowing for easy interaction with the application.

2. **Image Processing**: The application processes uploaded images using the `filtroHSV.py` script, which includes color space transformation and circle detection to extract relevant features from the images.

3. **Model Training**: The `entrenamiento.py` module implements a convolutional neural network (CNN) that is trained on the processed images, enabling the model to learn and classify coin quality.

4. **Prediction Generation**: After training, the model can analyze new images and generate predictions regarding the quality of the coins, which are displayed to the user in real-time.

5. **Visualization of Results**: The Streamlit interface provides visual feedback, displaying the uploaded images alongside the predicted quality class, enhancing user experience and understanding of the model's output.

- User Guide:
  - Instructions for running the prototype.

1. **Prerequisites**:

- Ensure you have Python installed on your system (preferably Python 3.6 or higher).

- Install the required libraries by running the following command in your terminal:

```
pip install tensorflow opencv-python numpy matplotlib seaborn streamlit
```

2. **Clone the Repository**:

- Clone the project repository to your local machine using:

```
git clone XXXXXX
```

- Navigate to the project directory:

```
cd XXXXX//XXXXX
```

3. **Prepare the Dataset**:

- Ensure that the dataset is organized correctly in the specified directories (e.g., 'Train' and 'Test' folders) as required by the `entrenamiento.py` script.

4. **Run the Streamlit Application**:

- Start the Streamlit application by executing the following command in the terminal:

```
streamlit run app.py
```

5. **Access the Application**:
   - Open your web browser and go to the URL provided in the terminal (usually `http://localhost:8501`).
   - You will see the user interface where you can upload images of coins.

6. **Interact with the Prototype**:
   - Upload images using the provided file uploader.
   - View the predictions and results displayed on the interface.

7. **Stop the Application**:
   - To stop the Streamlit application, return to the terminal and press `Ctrl + C`.

With these instructions, you will be able to run the prototype and interact with the AI-driven image analysis system.

- Brief description of the interface and its interactive components.

Final Presentation

- Content of the Presentation:
  - Brief introduction to the project and its relevance.

The user interface is built using Streamlit, providing a clean and intuitive layout for users to interact with the AI-driven image analysis system. Key components of the interface include:

1. **Title**: The interface prominently displays the title "Indicador de la Calidad de una Moneda," indicating the purpose of the application.

2. **Image Upload Component**: Users can upload images of coins through a file uploader. This component supports common image formats such as JPG, JPEG, and PNG, allowing for easy input of visual data.

3. **Image Display**: Once an image is uploaded, it is displayed on the interface, providing users with a visual confirmation of their selection.

4. **Prediction Output**: After processing the uploaded image, the interface displays the predicted quality class of the coin in a large, bold font. This output is generated by the trained model and provides immediate feedback to the user.

5. **Error Handling**: The interface includes error messages that inform users of any issues encountered during image upload or prediction generation, ensuring a smooth user experience.

Overall, the interface is designed to facilitate user engagement and provide clear, actionable insights based on the model's predictions.

- Description of the problem and the proposed solution.

The project addresses the challenge of assessing the quality of coins, which is traditionally a manual and subjective process. The proposed solution leverages artificial intelligence techniques, specifically convolutional neural networks (CNNs), to automate the analysis of coin images. By utilizing image processing methods and machine learning, the system aims to provide accurate and consistent quality assessments, enhancing efficiency and reliability in the evaluation process.

- Demonstration of the prototype in Streamlit.

The prototype is developed using Streamlit, offering an interactive user interface for coin quality assessment. Users can upload images of coins, which are then processed by the trained model to generate predictions regarding their quality. The interface displays the uploaded image alongside the predicted quality class, providing immediate feedback. This demonstration showcases the practical application of the AI model and its ability to assist users in evaluating coin quality.

- Conclusions and possible improvements.

The project successfully demonstrates the potential of AI-driven image analysis for automating coin quality assessment. The prototype effectively processes images and provides accurate predictions, highlighting the advantages of using machine learning in this domain. However, there are opportunities for improvement, such as enhancing the model's accuracy through additional training data, implementing more advanced image processing techniques, and expanding the user interface to include more features, such as batch processing of images or detailed analysis reports.

Future improvements for this work

There are many possibilities of improvements:

- Work the computer vision with images 3D
- Use readers of the coins with scanners 3D
- Self-improvement UPGRADE programmed using user data
- Normalize the images size
- Look for and use a better filter to found noise in the images

- Use the GPU target for more device capacity
- Use potential platforms like A
- Use different Cellphones (brands) to take images
- Take images with different quantities and colors light

Innovations in this project

- Starting an LLM model without a prior dataset

- Make a synthetic database since cero making own images with their respective labels using data augmentation.

- Training the AI with a synthetic dataset and working with data augmentation

- Training the AI model with the Few-Shot Learning method (40 epochs)

- Use the labels of the train images to give information, in this case is a number that indicates the proportion of white pixels respect the total image.

Monetization possibilities

- Fee for each scan using a mobile and/or web app - e.g., US$0.25

- Model rental via the web

- Fees for version upgrades

- Sale of specialized apps on other topics

Hardware used - Device Characteristics

- Operating System:
    - Name: Microsoft Windows 11 Home
    - Version: 10.0.26100 N/A Build 26100
    - Manufacturer: Microsoft Corporation
- System Information:
    - Manufacturer: Razer
    - Model: Blade 15 Advanced Model (Mid 2021) - RZ09-0409

    - System Type: x64-based PC
- Processor:
    - Processor(s): 1 Installed
    - Type: Intel64 Family 6 Model 141 Stepping 1 GenuineIntel
    - Speed: ~2496 MHz
- Memory:
    - Total Physical Memory: 32,516 MB
    - Available Physical Memory: 22,367 MB
    - Virtual Memory:
        - Max Size: 34,564 MB
        - Available: 22,715 MB
        - In Use: 11,849 MB
- BIOS:
    - Version: Razer 1.09
    - Date: 8/2/2021
- Network Cards:
    - Intel(R) Wi-Fi 6E AX210 160MHz (Media disconnected)
    - Realtek USB GbE Family Controller (DHCP Enabled)
- Virtualization:
    - Status: Running
    - Security Features Enabled: Various security properties including Secure Boot and DMA Protection.

Summarize:

- Operating System: Microsoft Windows 11 Home
- Processor: Intel64 Family 6 Model 141, ~2496 MHz
- Total Physical Memory: 32,516 MB
- Available Physical Memory: 22,367 MB
- Network Cards: Intel Wi-Fi 6E AX210 and Realtek USB GbE Family Controller

The GPU was not used in this project because the perform was enough.

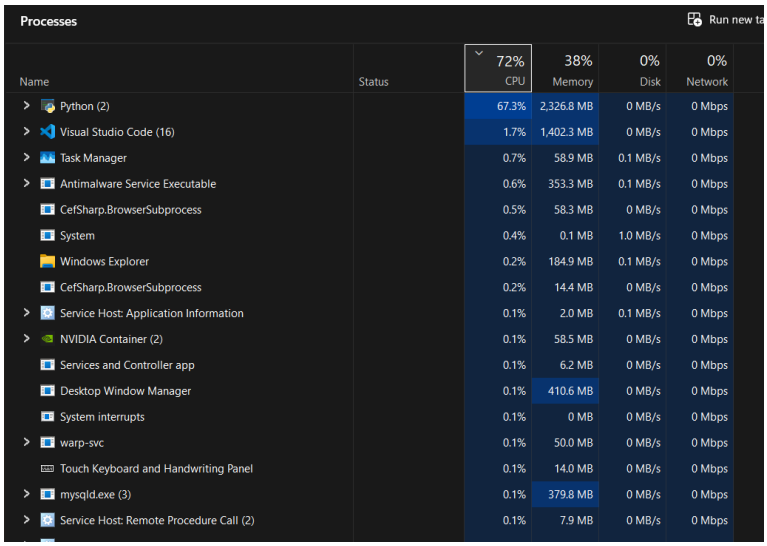
PERFORMANCE OF THE USED MACHINNE

Processes

Run new task

Name	Status	50% CPU	41% Memory	1% Disk	0% Network
Python (2)		46.2%	2,523.1 MB	0 MB/s	0 Mbps
Antimalware Service Executable		0.9%	354.9 MB	0.2 MB/s	0 Mbps
Task Manager		0.6%	60.2 MB	0.1 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.5%	57.3 MB	0 MB/s	0 Mbps
System		0.3%	0.1 MB	0.1 MB/s	0 Mbps
Service Host: Application Information		0.2%	2.0 MB	0.1 MB/s	0 Mbps
Windows Explorer		0.2%	149.2 MB	0.1 MB/s	0 Mbps
Visual Studio Code (19)		0.2%	2,153.8 MB	0 MB/s	0 Mbps
Service Host: State Repository Service		0.1%	23.8 MB	0 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.1%	14.4 MB	0 MB/s	0 Mbps
Desktop Window Manager		0.1%	541.5 MB	0 MB/s	0 Mbps
mysqld.exe (3)		0.1%	379.8 MB	0 MB/s	0 Mbps
NVIDIA Container (2)		0.1%	57.1 MB	0 MB/s	0 Mbps
DeepL		0.1%	199.9 MB	0 MB/s	0 Mbps
System interrupts		0.1%	0 MB	0 MB/s	0 Mbps
Shell Infrastructure Host		0.1%	6.2 MB	0 MB/s	0 Mbps
Microsoft Office SDX Helper		0.1%	4.2 MB	0 MB/s	0 Mbps
Touch Keyboard and Handwriting Panel		0.1%	13.7 MB	0 MB/s	0 Mbps
Service Host: CaptureService_30e9be		0%	1.4 MB	0 MB/s	0 Mbps
wsappx		0%	3.2 MB	0 MB/s	0 Mbps
SnippingTool.exe		0%	13.9 MB	0 MB/s	0 Mbps
Client Server Runtime Process		0%	2.3 MB	0 MB/s	0 Mbps

Performance of the device used in the moment of **filter process**.

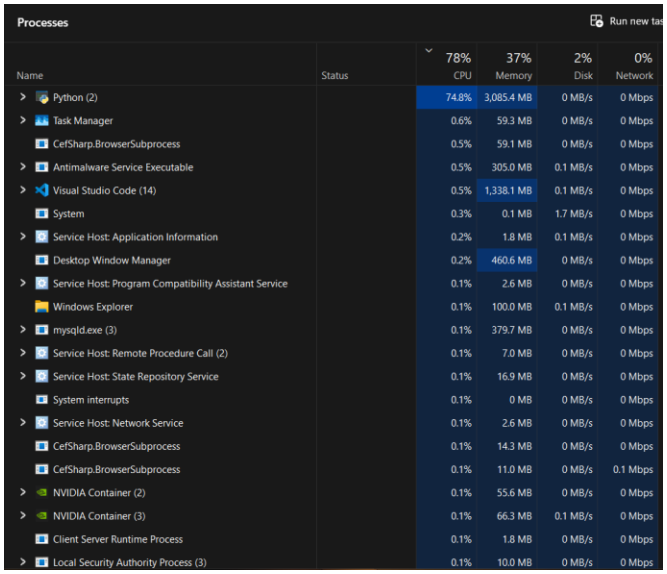


Processes — Run new task

Name	Status	72% CPU	38% Memory	0% Disk	0% Network
Python (2)		67.3%	2,326.8 MB	0 MB/s	0 Mbps
Visual Studio Code (16)		1.7%	1,402.3 MB	0 MB/s	0 Mbps
Task Manager		0.7%	58.9 MB	0.1 MB/s	0 Mbps
Antimalware Service Executable		0.6%	353.3 MB	0.1 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.5%	58.3 MB	0 MB/s	0 Mbps
System		0.4%	0.1 MB	1.0 MB/s	0 Mbps
Windows Explorer		0.2%	184.9 MB	0.1 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.2%	14.4 MB	0 MB/s	0 Mbps
Service Host: Application Information		0.1%	2.0 MB	0.1 MB/s	0 Mbps
NVIDIA Container (2)		0.1%	58.5 MB	0 MB/s	0 Mbps
Services and Controller app		0.1%	6.2 MB	0 MB/s	0 Mbps
Desktop Window Manager		0.1%	410.6 MB	0 MB/s	0 Mbps
System interrupts		0.1%	0 MB	0 MB/s	0 Mbps
warp-svc		0.1%	50.0 MB	0 MB/s	0 Mbps
Touch Keyboard and Handwriting Panel		0.1%	14.0 MB	0 MB/s	0 Mbps
mysqld.exe (3)		0.1%	379.8 MB	0 MB/s	0 Mbps
Service Host: Remote Procedure Call (2)		0.1%	7.9 MB	0 MB/s	0 Mbps

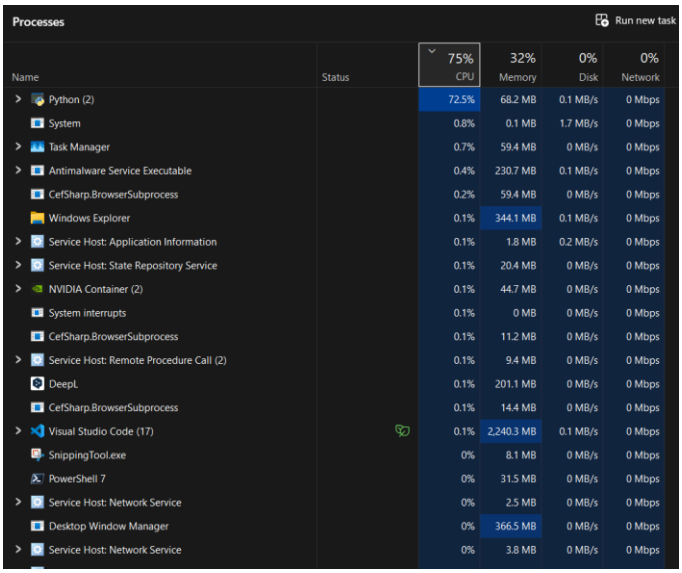
Processes

Name	Status	86% CPU	35% Memory	1% Disk	0% Network
Python (2)		80.4%	71.6 MB	0.1 MB/s	0 Mbps
System		3.5%	0.1 MB	1.6 MB/s	0 Mbps
Visual Studio Code (20)		0.7%	2,070.5 MB	0 MB/s	0 Mbps
Task Manager		0.6%	74.3 MB	0.1 MB/s	0 Mbps
Antimalware Service Executable		0.3%	234.3 MB	0.1 MB/s	0 Mbps
Service Host: Application Information		0.1%	2.0 MB	0.2 MB/s	0 Mbps
Windows Explorer		0.1%	724.9 MB	0.1 MB/s	0 Mbps
warp-svc		0.1%	52.2 MB	0 MB/s	0 Mbps
NVIDIA Container (3)		0.1%	57.1 MB	0.1 MB/s	0 Mbps
System interrupts		0.1%	0 MB	0 MB/s	0 Mbps
Microsoft Office SDX Helper		0.1%	5.1 MB	0 MB/s	0 Mbps
Desktop Window Manager		0.1%	185.1 MB	0 MB/s	0 Mbps
DeepL		0.1%	201.7 MB	0 MB/s	0 Mbps
Service Host: Remote Procedure Call (2)		0.1%	7.4 MB	0 MB/s	0 Mbps
Microsoft Network Realtime Inspection Service		0.1%	3.1 MB	0 MB/s	0 Mbps
Client Server Runtime Process		0.1%	2.5 MB	0 MB/s	0 Mbps
conhost.exe		0%	0.6 MB	0 MB/s	0 Mbps
docker.exe		0%	1.1 MB	0 MB/s	0 Mbps
SnippingTool.exe		0%	1.9 MB	0 MB/s	0 Mbps
Service Host: Capability Access Manager Service		0%	4.6 MB	0.1 MB/s	0 Mbps
Windows Driver Foundation - User-mode Driver Framew...		0%	3.7 MB	0 MB/s	0 Mbps
Service Host: State Repository Service		0%	11.3 MB	0 MB/s	0 Mbps
NVIDIA Container (3)		0%	58.7 MB	0 MB/s	0 Mbps
Service Host: Local System (Network Restricted)		0%	5.7 MB	0 MB/s	0 Mbps
Service Host: Network Service		0%	2.4 MB	0 MB/s	0 Mbps



Processes — Run new task

Name	Status	78% CPU	37% Memory	2% Disk	0% Network
Python (2)		74.8%	3,085.4 MB	0 MB/s	0 Mbps
Task Manager		0.6%	59.3 MB	0 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.5%	59.1 MB	0 MB/s	0 Mbps
Antimalware Service Executable		0.5%	305.0 MB	0.1 MB/s	0 Mbps
Visual Studio Code (14)		0.5%	1,338.1 MB	0.1 MB/s	0 Mbps
System		0.3%	0.1 MB	1.7 MB/s	0 Mbps
Service Host: Application Information		0.2%	1.8 MB	0.1 MB/s	0 Mbps
Desktop Window Manager		0.2%	460.6 MB	0 MB/s	0 Mbps
Service Host: Program Compatibility Assistant Service		0.1%	2.6 MB	0 MB/s	0 Mbps
Windows Explorer		0.1%	100.0 MB	0.1 MB/s	0 Mbps
mysqld.exe (3)		0.1%	379.7 MB	0 MB/s	0 Mbps
Service Host: Remote Procedure Call (2)		0.1%	7.0 MB	0 MB/s	0 Mbps
Service Host: State Repository Service		0.1%	16.9 MB	0 MB/s	0 Mbps
System interrupts		0.1%	0 MB	0 MB/s	0 Mbps
Service Host: Network Service		0.1%	2.6 MB	0 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.1%	14.3 MB	0 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.1%	11.0 MB	0 MB/s	0.1 Mbps
NVIDIA Container (2)		0.1%	55.6 MB	0 MB/s	0 Mbps
NVIDIA Container (3)		0.1%	66.3 MB	0.1 MB/s	0 Mbps
Client Server Runtime Process		0.1%	1.8 MB	0 MB/s	0 Mbps
Local Security Authority Process (3)		0.1%	10.0 MB	0 MB/s	0 Mbps



Processes — Run new task

Name	Status	75% CPU	32% Memory	0% Disk	0% Network
Python (2)		72.5%	68.2 MB	0.1 MB/s	0 Mbps
System		0.8%	0.1 MB	1.7 MB/s	0 Mbps
Task Manager		0.7%	59.4 MB	0 MB/s	0 Mbps
Antimalware Service Executable		0.4%	230.7 MB	0.1 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.2%	59.4 MB	0 MB/s	0 Mbps
Windows Explorer		0.1%	344.1 MB	0.1 MB/s	0 Mbps
Service Host: Application Information		0.1%	1.8 MB	0.2 MB/s	0 Mbps
Service Host: State Repository Service		0.1%	20.4 MB	0 MB/s	0 Mbps
NVIDIA Container (2)		0.1%	44.7 MB	0 MB/s	0 Mbps
System interrupts		0.1%	0 MB	0 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.1%	11.2 MB	0 MB/s	0 Mbps
Service Host: Remote Procedure Call (2)		0.1%	9.4 MB	0 MB/s	0 Mbps
DeepL		0.1%	201.1 MB	0 MB/s	0 Mbps
CefSharp.BrowserSubprocess		0.1%	14.4 MB	0 MB/s	0 Mbps
Visual Studio Code (17)		0.1%	2,240.3 MB	0.1 MB/s	0 Mbps
SnippingTool.exe		0%	8.1 MB	0 MB/s	0 Mbps
PowerShell 7		0%	31.5 MB	0 MB/s	0 Mbps
Service Host: Network Service		0%	2.5 MB	0 MB/s	0 Mbps
Desktop Window Manager		0%	366.5 MB	0 MB/s	0 Mbps
Service Host: Network Service		0%	3.8 MB	0 MB/s	0 Mbps

Performance in the moment of **training process**.

Vocabulary

Automated Assessment: The use of technology to evaluate and determine the quality of an item without human intervention.

Convolutional Neural Networks (CNNs): A class of deep neural networks, most commonly applied to analyzing visual imagery.

Data Augmentation: Techniques used to increase the diversity of the training dataset by applying random transformations to the training images.

Denomination: The face value or the value assigned to a coin, which indicates its worth in terms of currency. For example, a coin may have a denomination of 1 cent, 5 cents, 1 dollar, etc.

Digital Transformation: The integration of digital technology into all areas of a business, fundamentally changing how it operates and delivers value to customers.

Grading Standards: Established criteria used to assess the quality and condition of coins.

Image Processing: A method to perform operations on an image to enhance it or extract useful information.

Numismatic: Relating to or consisting of coins, paper currency, and medals, and the study or collection of these items.

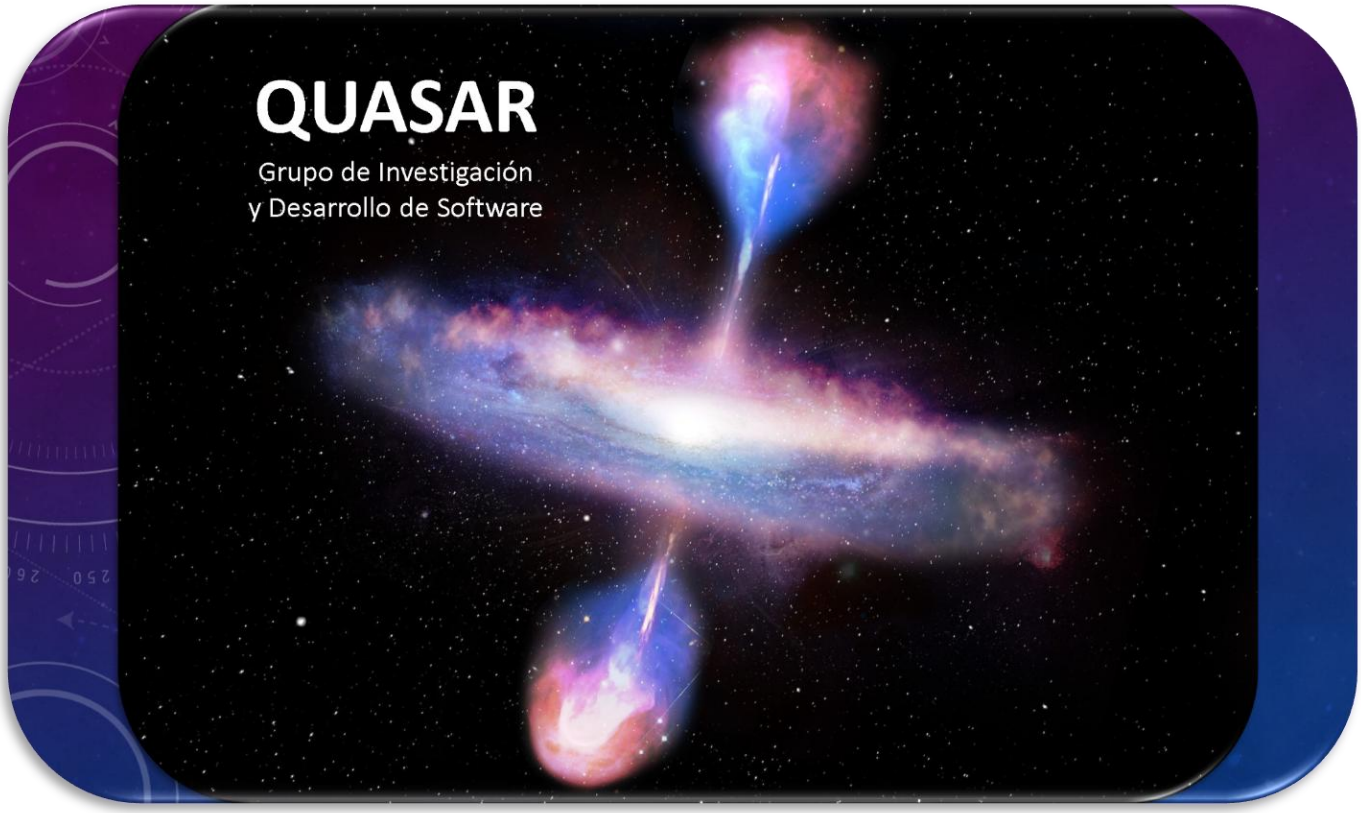
QUASAR: In astronomy, a quasar (short for "quasi-stellar object") is an extremely luminous and distant celestial object powered by a supermassive black hole at the center of a galaxy. Quasars emit enormous amounts of energy, often outshining entire galaxies, and are considered some of the brightest objects in the universe.

Segmentation: The process of partitioning an image into multiple segments to simplify its representation.

Thresholding: A technique in image processing to create binary images from grayscale images.

Software, document and video REPOSITORIES:

https://github.com/CantandoBien/TalentoTech-IA



Medellín – Colombia.